

Patribots (FRC 4738) — Four-Year Code Sophistication Scoring

Scored against `frc-code-sophistication-rubric.md`. Unlike the earlier corpus run (shallow clones, `.git` stripped), these are **full clones with complete history**, pulled from `github.com/Patribots4738`:

Repo	Season	Commits	Java files	Last commit
RapidReact2022	2022	92	53	2022-11-12
ChargedUp2023	2023	1,095	19	2023-11-19
Crescendo2024	2024	1,727	61	2024-11-08
Reefscape2025	2025	632	67	2025-11-29

Every level below was confirmed by opening the files behind the grep hits, not presence alone. Half-steps used where a repo sits between anchors.

Scoresheet

Season	D1 Arch	D2 Coord	D3 Sim	D4 Test	D5 Log	D6 Auto	D7 Vision	D8 Sustain	Σ/32
2022 RapidReact	1	0.5	0	0	1	1	1	0.5	5.0
2023 ChargedUp	1.5	1	0	0	1	2.5	2	2	10.0
2024 Crescendo	3	2	1.5	0	3	3	2.5	2.5	17.5
2025 Reefscape	3	3	2	0	3	3	3	3	20.0

A clean monotonic climb — **5 → 10 → 17.5 → 20** — with the entire jump concentrated in one offseason (2023→2024). One number never moves: **D4 (testing) = 0, all four years**.

(Note: the prior corpus master CSV listed 2025 at 19.5 with D2 = 2.5. This re-score puts D2 at 3 — see the 2025 notes — so the independently confirmed total is 20.0.)

Per-year profiles

2022 RapidReact — 5.0 — "pre-framework"

Not command-based at all. `Robot.java` extends `TimedRobot` with custom device wrappers (`Motor` , `MotorGroup` , `Falcon` , `Gamepad` , `Turret`); joystick values flow to motor groups inside the periodic loop. No `SubsystemBase` , no command composition, no README, no CI. The one forward-looking piece is a vision-aimed turret (Limelight `tx/ty` targeting across 15 files) — targeting only, no pose estimation. This is the baseline the rest is measured against.

2023 ChargedUp — 10.0 — "custom framework, strong on the field"

Still **not** WPILib command-based (zero `SubsystemBase` / `CommandBase`); Patribots ran their own architecture organized into `hardware/` , `calc/` , `auto/` packages with a `hardware/Swerve` class. What's striking is how much capability they reached *without* the modern scaffolding: PhotonVision AprilTag pose estimation feeding `addVisionMeasurement` into a `SwerveDrivePoseEstimator` , with dual-camera ambiguity comparison for rejection (`PhotonCameraUtil` , `AutoAlignment` drive-to-pose), plus 45 PathPlanner path files and `AutoBuilder` autos (D6 = 2.5). Logging is still `SmartDashboard` -only; no sim, no tests. README + CI (build) arrive (D8 = 2). **The story of this repo is good results on a soon-to-be-replaced foundation.**

2024 Crescendo — 17.5 — "two robots in one repo"

The discontinuity — but the commit history shows the score is half in-season, half offseason rebuild, and the rubric scores the final state. During the actual 2024 competition season (Jan–April) this repo was command-based on the **Monologue** logging library with no IO layer (first `interface GyroIO` lands 2024-08-21; switch to `loggedrobot` is 2024-07-30). The IO-layer + AdvantageKit architecture that earns the D1/D5 = 3 below was built after the season, July–August 2024, on the same repo. So 17.5 is the repo's final state; the 2024 competition robot was closer to ~12 (D1 ≈ 1, D5 ≈ 2). With that caveat, the final state scores: - **D1 = 3**: per-subsystem *IO interfaces across all eleven mechanisms (`GyroIO`, `ShooterIO`, `PivotIO`, `IntakeIO`, `IndexerIO`, `ClimbIO`, `ElevatorIO`, `AmpperIO`, `LimelightIO`, `PicoColorSensorIO`, `MAXSwerveModuleIO`) with real implementations behind each. - **D5 = 3**: full AdvantageKit — `@AutoLog` inputs structs and `Logger.processInputs` across the subsystem set (35 hits). - **D6 = 3**: PathPlanner (87 paths) plus Choreo actually wired (`ChoreoStorage` calls `PathPlannerPath.fromChoreoTrajectory`) plus `LocalADStar` on-the-fly pathfinding. This is their autonomous high-water mark. - **D2 = 2**: coordination via command-manager classes (`PieceControl`, `ShooterCmds`, `ShooterCalc`) that fan piece-handling and aiming across subsystems — richer than plain composition, but no explicit state-machine/superstructure with guarded transitions yet. - **D3 = 1.5**: `simulationPeriodic` plus a `NeoPhysicsSim` / `DCMotorSim` at the motor-wrapper level — generic and partial, not mechanism-specific physics in an IO-sim implementation. - **D4 = 0**, D7 = 2.5, D8 = 2.5.

2025 Reefscape — 20.0 — "consolidation + a real coordinator"

Refinement of the 2024 platform rather than another leap: - **D1 = 3**: the IO seam matures into dual real implementations — `*IOKraken` and `*IONeo` for the same interface (Elevator, Wrist, Climb, Coral/Algae claw, Module), 11 implementation files. Cleaner hardware portability than 2024. - **D2 = 3 (up from 2024's 2)**: a genuine `Superstructure` coordinator. `SuperState` objects bundle a robot-wide goal into per-subsystem states (`ArmState` / `ClawState` / `ClimbState`); a `targetState` is the requested intent, and transitions are guarded (`() -> elevator.atPosition(...)` conditions, `waitUntil` gates). Intent is separated from execution — that's the level-3 anchor. - **D7 = 3**: two Limelights behind a `VisionIO` interface, MegaTag2, dynamic `setVisionMeasurementStdDevs` and rejection feeding `addVisionMeasurement` — fused, filtered, multi-camera. - **D5 = 3** (AdvantageKit `processInputs` across 8 subsystems), **D6 = 3** (252 PathPlanner paths + `LocalADStarAK` on-the-fly + `Alignment` drive-to-pose; note Choreo from 2024 doesn't clearly reappear), **D8 = 3** (CI + README + a recurring `util/` library carried across seasons). - **D3 = 2**, **D4 = 0**.

Within-season trajectories (from the commit history)

Every season has the same heartbeat — a January kickoff spike, heavy February/March build-and-compete, an April taper, then a fall bump that is not rewrite work but **Beach Blitz** (the San Diego offseason event; the late-Oct/early-Nov commits are all "day 1 bb," "Beach blitz," competition tuning). The architecture decisions, by contrast, happen at kickoff or in deep summer. What each year was actually doing:

2022 (92 commits, 4 contributors). Kickoff is raw: "Initial Comp Code," a hand-rolled motor position/speed logger, "WORKS NOW," "?" Build season is Jan–March on the custom `TimedRobot` structure; fall is hardcoded-distance autos and turret-zeroing for Beach Blitz ("Hardcode Distances Auto," "desperate attempt to get consistency"). A small team brute-forcing a working robot.

2023 (1,095 commits, 10 contributors). The telling move is the *first week*: "Convert command to iterative" and "Refactor DriveSubsystem.java to Swerve.java **Delete RobotContainer.java**" — they started from a command-based swerve template and deliberately tore out WPILib's command framework to run their own iterative architecture. Vision arrives mid-season (PhotonVision pose estimation, 2023-02-07). The commit volume triples February–March (282→318/month): this is the year the program scales up its contributor base and its on-field sophistication, all on a framework they'd ultimately abandon.

2024 (1,727 commits, 10+ contributors — their highest-volume year). Kickoff *reverses* 2023: "Remove OICalc, since we use command based" — back to command-based, with **Monologue** logging and Choreo modular autos wired by 2024-01-25. They compete the whole season this way. Then the inflection, entirely in summer: 2024-07-30 "switch to loggedrobot and basic setup before big io division," 2024-08-02 "ampper adapted to adv kit," 2024-08-21 "gyro???" (first IO interface). The 676-commit February is in-season grind; the August work is the architectural rebuild that defines everything after.

2025 (632 commits, 10+ contributors). Starts already modern — 2025-01-10 "add claw with akit and logged constants, add loggedtunableboolean." The season's intellectual work is coordination: "add superstructure control" (2025-01-12) and a deliberate "start state implementation" (2025-02-12) that becomes the `SuperState` machine. Fewer commits than 2024 because less was being invented — this is a refinement year on a stable base, culminating in a winning Beach Blitz ("We f***ing won," 2025-11-04) and post-event polish still landing in late November ("cosine compensation to minimize skew... tested on the real robot ").

The four-year constant, verified against all history: `git log --all -S"@Test"` returns nothing. Not one unit test has been written on any branch of any season repo, ever. The D4 = 0 column isn't a snapshot — it's a four-year fact.

The shape of four years

Two things stand out when you read the profile across time rather than down a column.

1. The leap happened in the 2024 offseason, not in any competition season. From 2022 to 2023 the Patribots improved *within* a custom, non-command-based framework — better vision, better autos, same architectural ceiling. The leap to elite-track scores came from a clean-sheet adoption of the IO-layer + AdvantageKit stack in **July–August 2024** — between the Crescendo season and the Reefscape season, on the Crescendo repo, not during a build season. That's why Reefscape2025 *opens* (its 2025-01-10 commit) already on AdvantageKit with logged constants: the foundation was a finished offseason project before kickoff. The pattern across four years is that **architecture is decided when there's no game to play, and refined under competition pressure** — they are a team that paid the architectural cost most teams never pay, and paid it in the summer.

2. They built every seam and are sitting on two uncollected dividends. The elite-track architecture doc frames advanced capability as add-ons that attach to three seams: the **IO seam** (D1), the **coordination seam** (D2), and the **state seam** (D7/RobotState). Patribots have built the IO seam (2024) and the coordination seam (2025 Superstructure). What they have *not* done is collect the two dividends that those seams make nearly free:

- **Testing (D4) has been 0 for four straight years** — and D4 is the single rarest marker in the entire San Diego corpus, the clearest signal of real software-engineering culture. They own the exact infrastructure (`@AutoLog` inputs structs, per-subsystem IO interfaces) that the doc says makes unit testing "a deferred dividend you populate, not a refactor you rebuild."
- **Simulation (D3) has never cleared ~2**, stuck at a generic `DCMotorSim` buried in the motor wrappers (and partly commented out), instead of mechanism physics in a proper `XxxIOSim`.

These aren't independent gaps — they're the *same* gap. Both attach to the IO seam they already cut.

What to work on next — in priority order

1. Collect the testing dividend (D4: 0 → 2). Highest leverage, lowest new architecture.

This is the move. Four years at zero on the corpus's rarest dimension, while owning the infrastructure that makes it cheap. Concretely, per the elite-arch doc's §2.2–2.3:

- Add a real `XxxIOSim` implementation for **one** mechanism — start with the elevator or wrist (clean 1-DOF physics). Use WPILib `ElevatorSim` / `SingleJointedArmSim` inside the IO's `updateInputs`, looping *above the line* (`setVoltage`) so the sim and real share the subsystem's one controller and stay in parity.
- Write **one** unit test that constructs that subsystem with its `IOSim`, runs it, and asserts the mechanism reaches a setpoint.
- Make CI run it: their `gradle.yml` currently runs `./gradlew build`, not `test`. Change one line so the test gates merges → that simultaneously moves toward D4 = 3.

The inputs-struct style they already use (AdvantageKit `@AutoLog`) is exactly what makes this an *addition* at a known attachment point rather than a rewrite.

2. Promote simulation from the motor wrapper to the IO layer (D3: 2 → 3).

Same seam, same work as #1. Filling `XxxIOSim` with mechanism physics *is* the simulation upgrade — do the elevator/wrist sims for testing and D3 rises with D4. Then add a `SIM` / `REPLAY` mode switch at the single subsystem-construction point (the doc's "selection point") so they get AdvantageKit **replay** of real matches for free (a D5 → 4 path).

3. Build the state seam — a `RobotState` world model (D7: 3 → 4).

The one seam they haven't cut. Today the pose estimator lives privately inside `Swerve`. Extract a dedicated `RobotState` that owns the `SwerveDrivePoseEstimator` (and later a `TimeInterpolatableBuffer<Pose2d>` history + game-piece state); subsystems feed it, vision corrects it, pathing and decisions read from it. That's the difference between "pose estimation exists" (level 3) and "a world model is the architecture" (level 4), and it's a refactor they're now architecturally ready for.

When to do it: their own history says summer. The IO layer was a July–August 2024 offseason project; the Superstructure was a kickoff project. Testing and IO-level simulation are exactly the kind of foundational, no-game-on-the-line work they've twice shown they do best between seasons. The summer of 2026 is the slot — and unlike 2024's rebuild, this one requires no demolition, only filling seams that already exist.

If they do exactly one thing: write the first unit test (step 1). It's the rarest, longest-standing gap — four years and zero attempts in the entire commit history — it requires the least new architecture because they already built the seam, and it's the clearest available signal that 4738 is an engineering program and not just a strong-results team.